

DV - Optimizing IT Support Team Performance Using Analytics (Supportlytics)

Milestone 1 - Week 1 & 2 Module 1

1. Objectives

This project analyzes IT support ticket data to uncover performance trends, optimize resolution times, and improve overall service efficiency. The analysis will translate raw ticket-level data into actionable operational recommendations for IT support leadership.

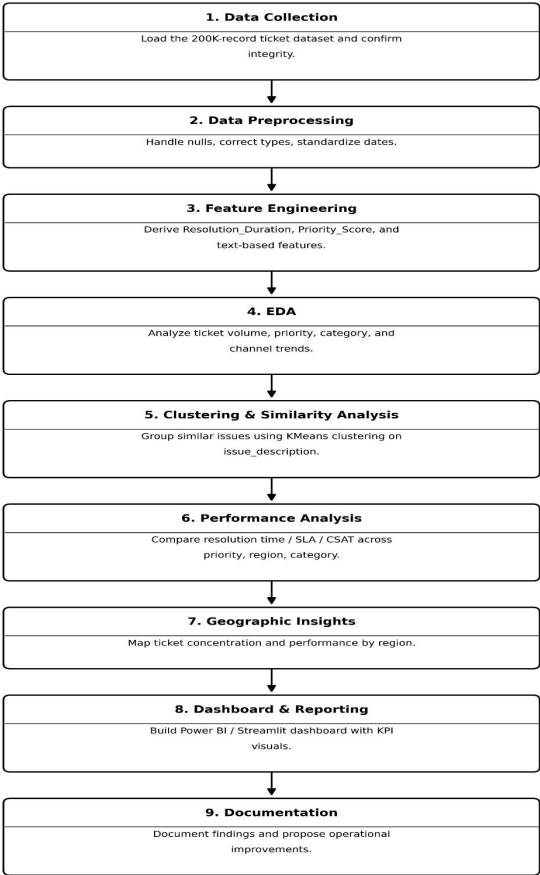
- Understand and process the IT support ticket dataset.
- Explore trends in ticket volume, priority, category, and channel distribution.
- Identify clusters of similar issues using text similarity and clustering techniques applied to issue descriptions.
- Quantify support performance through resolution time, first-response time, SLA compliance, and escalation behavior.
- Visualize performance metrics through dashboards and charts.
- Summarize the final analytical report and presentation.

2. KPIs and Workflow

2.1 Key Performance Indicators

- **Average Resolution Time:** mean time between ticket creation and resolution/closure
- **First Response Time:** average time to the first agent reply
- **Ticket Volume by Category/Type:** count of tickets per issue category (billing, technical, account, etc.)
- **Ticket Volume by Priority:** distribution across low/medium/high/urgent
- **Ticket Volume by Channel:** split across email, chat, phone, social media
- **SLA Compliance Rate:** % of tickets resolved within target time
- **Customer Satisfaction Score:** average rating if a satisfaction field exists
- **Backlog / Open Ticket Rate:** proportion of tickets still open/pending vs. closed

2.2 Workflow



3. Load the CSV Dataset Using Pandas

The dataset was loaded and using pandas to confirm structure.

Code:

```
import pandas as pd
df = pd.read_csv("./customer_support_tickets_200k.csv")
df.head()
```

	ticket_id	customer_name	customer_email	product	category	issue_description	resolution_notes	priority	status	channel	...
0	1	Patricia Smith	patricia.smith760@outlook.com	Web Portal	Account Suspension	The payment was deducted from my bank account ...	Data synchronization restored after backend se...	Urgent	Open	Email	...
1	2	Patricia Williams	patricia.williams390@gmail.com	Mobile App	Performance Issue	I found a bug in the latest update affecting r...	Provided step-by-step troubleshooting instruct...	Urgent	Closed	Email	...
2	3	William Anderson	william.anderson651@outlook.com	Web Portal	Performance Issue	The application crashes whenever I try to uplo...	Provided step-by-step troubleshooting instruct...	Medium	Closed	Chat	...
3	4	David Miller	david.miller672@icloud.com	Payment Gateway	Subscription Cancellation	My subscription was cancelled without my requ...	Provided step-by-step troubleshooting instruct...	Medium	Closed	Social Media	...
4	5	Robert Gonzalez	robert.gonzalez391@hotmail.com	Web Portal	Feature Request	The system is not syncing data across devices ...	We have reset the account credentials and advl...	High	Pending Customer	Email	...

The dataset was loaded into a Pandas DataFrame using `read_csv()`. The `head()` function was then used to display the first few records and verify that the dataset was loaded correctly and has the expected structure.

4. Explore schema & data types

Code:

```
df.info()
df['ticket_created_date'] = pd.to_datetime(df['ticket_created_date'])
df['ticket_resolved_date'] = pd.to_datetime(df['ticket_resolved_date'])
df[['ticket_created_date', 'ticket_resolved_date']].dtypes
```

Before Dtype Changed:

```
<class 'pandas.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 30 columns):
#   Column              Non-Null Count  Dtype
---  -
0   ticket_id            200000 non-null int64
1   customer_name        200000 non-null str
2   customer_email       200000 non-null str
3   product              200000 non-null str
4   category             200000 non-null str
5   issue_description    200000 non-null str
6   resolution_notes     200000 non-null str
7   priority             200000 non-null str
8   status              200000 non-null str
9   channel              200000 non-null str
10  region               200000 non-null str
11  customer_age         200000 non-null int64
12  customer_gender      200000 non-null str
13  subscription_type    200000 non-null str
14  customer_tenure_months 200000 non-null int64
15  previous_tickets     200000 non-null int64
16  customer_satisfaction_score 200000 non-null int64
17  first_response_time_hours 200000 non-null float64
18  resolution_time_hours 200000 non-null float64
19  ticket_created_date  200000 non-null str
...
28  issue_complexity_score 200000 non-null int64
29  customer_segment     200000 non-null str
dtypes: float64(2), int64(6), str(22)
memory usage: 45.8 MB
```

After Dtype Changed:

```
ticket_id            int64
customer_name        str
customer_email       str
product              str
category             str
issue_description    str
resolution_notes     str
priority             str
status              str
channel              str
region              str
customer_age         int64
customer_gender      str
subscription_type    str
customer_tenure_months  int64
previous_tickets     int64
customer_satisfaction_score  int64
first_response_time_hours  float64
resolution_time_hours  float64
ticket_created_date  datetime64[us]
ticket_resolved_date  datetime64[us]
escalated            str
sla_breached         str
operating_system     str
browser              str
payment_method       str
language             str
preferred_contact_time  str
issue_complexity_score  int64
customer_segment     str
```

The dataset schema and data types were first checked using `df.info()` to identify the columns, number of records, and their existing data types. The `ticket_created_date` and `ticket_resolved_date` columns were then converted into **datetime format**, which allows us to accurately calculate resolution time, compare dates, and perform time-based analysis.

Before Data Type Changed: Date columns were stored as object (text/string).

After Data Type Changed: Date columns were converted to `datetime64[ns]`, making them ready for date and time calculations.

5. missing values

Code:

```
df.isnull().sum()
```

```

ticket_id      0
customer_name  0
customer_email 0
product        0
category       0
issue_description
resolution_notes
priority       0
status        0
channel        0
region         0
customer_age   0
customer_gender
subscription_type
customer_tenure_months
previous_tickets
customer_satisfaction_score
first_response_time_hours
resolution_time_hours
ticket_created_date
ticket_resolved_date
escalated      0
sla_breached   0
operating_system
browser        40023
...
language       0
preferred_contact_time
issue_complexity_score
customer_segment
dtype: int64

```

The `df.isnull().sum()` function was used to identify the number of missing values present in each column of the dataset. This step helps determine whether any data is incomplete and needs to be handled before performing further analysis or building machine learning models.

6. Calculate initial ticket distribution by Type, Priority, and Category

Note: The values are generated after the dataset is cleaned.

After cleaning the dataset, the ticket distribution was analyzed based on **Type, Priority, and Category**. This helps understand how tickets are distributed across different support types and identify which ticket types occur most frequently.

6.1 Ticket volume by Type:

Code:

```

print('--- Distribution by Type ---')
display(df['type'].value_counts().rename_axis('Type').reset_index(name='Count'))

```

	Type	Count
0	Request	80029
1	Problem	60052
2	Incident	59919

The `value_counts()` function was used to count the number of tickets belonging to each ticket type. The results were then formatted into a table using `reset_index()` for easier interpretation and analysis.

6.2 Ticket volume by Priority:

Code:

```

print('--- Distribution by Priority ---')
display(df['priority'].value_counts().rename_axis('Priority').reset_index(name='Count'))

```

	Priority	Count
0	High	50241
1	Urgent	50143
2	Medium	49854
3	Low	49762

The `value_counts()` function was used to calculate the number of tickets for each **priority level**, such as High, Medium, and Low. This helps identify which priority level has the highest number of customer support tickets and understand the overall workload distribution.

6.3 Ticket volume by Category:

Code:

```

print('--- Distribution by Category ---')
display(df['category'].value_counts().rename_axis('Category').reset_index(name='Count'))

```

	Category	Count
0	Feature Request	20169
1	Subscription Cancellation	20096
2	Performance Issue	20074
3	Security Concern	20040
4	Login Issue	20002
5	Payment Problem	19997
6	Bug Report	19981
7	Refund Request	19900
8	Data Sync Issue	19877
9	Account Suspension	19864

The same approach was used to calculate the number of tickets in each support category. This helps identify the most frequently reported categories and provides a clear understanding of the areas generating the highest volume of customer support requests.

Module 2

1. Handle missing or incorrect data in text fields

Code:

```
df.drop(columns=['browser'], inplace=True)
print('browser' in df.columns)
print(df.shape)
```

```
False
(200000, 29)
```

The browser column was removed from the dataset using drop() because it was not required for further analysis or contained unnecessary/incorrect information. The inplace=True parameter permanently applies the change to the DataFrame. The print() statements were then used to verify that the browser column was successfully removed and to check the updated number of rows and columns in the dataset.

2. Create new features: Type

Code:

```
category_type_map = {
    'Account Suspension': 'Request',
    'Bug Report': 'Problem',
    'Data Sync Issue': 'Incident',
    'Feature Request': 'Request',
    'Login Issue': 'Incident',
    'Payment Problem': 'Problem',
    'Performance Issue': 'Problem',
    'Refund Request': 'Request',
    'Security Concern': 'Incident',
    'Subscription Cancellation': 'Request'
}
cat_idx = df.columns.get_loc('category')
df.insert(cat_idx + 1, 'type', df['category'].map(category_type_map))
df[['category', 'type']].head(10)
```


	category	type
0	Account Suspension	Request
1	Performance Issue	Problem
2	Performance Issue	Problem
3	Subscription Cancellation	Request
4	Feature Request	Request
5	Payment Problem	Problem
6	Security Concern	Incident
7	Subscription Cancellation	Request
8	Security Concern	Incident
9	Performance Issue	Problem

A new **type** feature was created by mapping each ticket category to a broader ticket type such as **Request, Problem, or Incident**. This helps group similar support issues together and makes it easier to analyze the overall nature of customer tickets. The `map()` function applies the predefined category-to-type mapping, and the new type column is inserted immediately after the category column. Finally, the first 10 records were displayed to verify that the mapping was performed correctly.

3. Create new features: Resolution_Duration and Priority_Score

Code:

```
df['ticket_created_date'] = pd.to_datetime(df['ticket_created_date'])
df['ticket_resolved_date'] = pd.to_datetime(df['ticket_resolved_date'])
df['resolution_duration'] = (df['ticket_resolved_date'] - df['ticket_created_date']).dt.days
```

```
priority_score_map = {'Low': 1, 'Medium': 2, 'High': 3, 'Urgent': 4}
df['priority_score'] = df['priority'].map(priority_score_map)
```

```
df[['priority', 'priority_score', 'ticket_created_date', 'ticket_resolved_date', 'resolution_duration']].head(10)
```

	priority	priority_score	ticket_created_date	ticket_resolved_date	resolution_duration
0	Urgent	4	2023-05-17	2023-05-20	3
1	Urgent	4	2024-01-06	2024-01-19	13
2	Medium	2	2022-11-30	2022-12-05	5
3	Medium	2	2024-03-21	2024-04-04	14
4	High	3	2024-08-16	2024-08-24	8
5	Low	1	2024-09-27	2024-09-27	0
6	Medium	2	2024-02-04	2024-02-11	7
7	Low	1	2024-08-26	2024-08-29	3
8	Low	1	2024-06-26	2024-07-08	12
9	Urgent	4	2022-05-05	2022-05-16	11

Two new features, **resolution_duration** and **priority_score**, were created to improve the analysis of customer support tickets. The **resolution_duration** feature calculates the number of days taken to resolve each ticket by finding the difference between the ticket creation and resolution dates.

The **priority_score** feature converts the priority levels into numerical values: **Low = 1, Medium = 2, High = 3, and Urgent = 4**.

4. This makes the priority information easier to use for statistical analysis and machine learning models.

4. Create new features: Queue

```
queue_map = {
    'Account Suspension': 'Account Management',
    'Bug Report': 'Development Team',
    'Data Sync Issue': 'Technical Support',
    'Feature Request': 'Product Team',
    'Login Issue': 'Technical Support',
    'Payment Problem': 'Billing Support',
    'Performance Issue': 'Infrastructure Team',
    'Refund Request': 'Billing Support',
    'Security Concern': 'Security Team',
    'Subscription Cancellation': 'Account Management'
}
df['queue'] = df['category'].map(queue_map)
df[['category', 'queue']].head(10)
```

	category	queue
0	Account Suspension	Account Management
1	Performance Issue	Infrastructure Team
2	Performance Issue	Infrastructure Team
3	Subscription Cancellation	Account Management
4	Feature Request	Product Team
5	Payment Problem	Billing Support
6	Security Concern	Security Team
7	Subscription Cancellation	Account Management
8	Security Concern	Security Team
9	Performance Issue	Infrastructure Team

A new **queue** feature was created to assign each customer support ticket to the appropriate **team or department** based on its category. For example, payment and refund-related issues are assigned to the **Billing Support** queue, while security-related issues are assigned to the **Security Team**.

The `map()` function was used to match each ticket category with its corresponding queue. The first 10 records were displayed to verify that the mapping was applied correctly.

5. Exported the cleaned dataset

Code:

```
df.to_csv('./cleaned_customer_support_tickets_200k.csv', index=False)
print('Exported successfully:', df.shape)
```

```
Exported successfully: (200000, 32)
```

After completing the data cleaning and feature engineering steps, the processed dataset was exported as a new CSV file named `cleaned_customer_support_tickets_200k.csv`. The `index=False` parameter ensures that the Pandas index is not added as an extra column in the output file.

Milestone 2 - Week 3 & 4

Module 3

Exploratory Visualization

- Plot ticket type distribution
- Identify top categories and clusters by frequency
- Visualize tickets by priority and queue

Code:

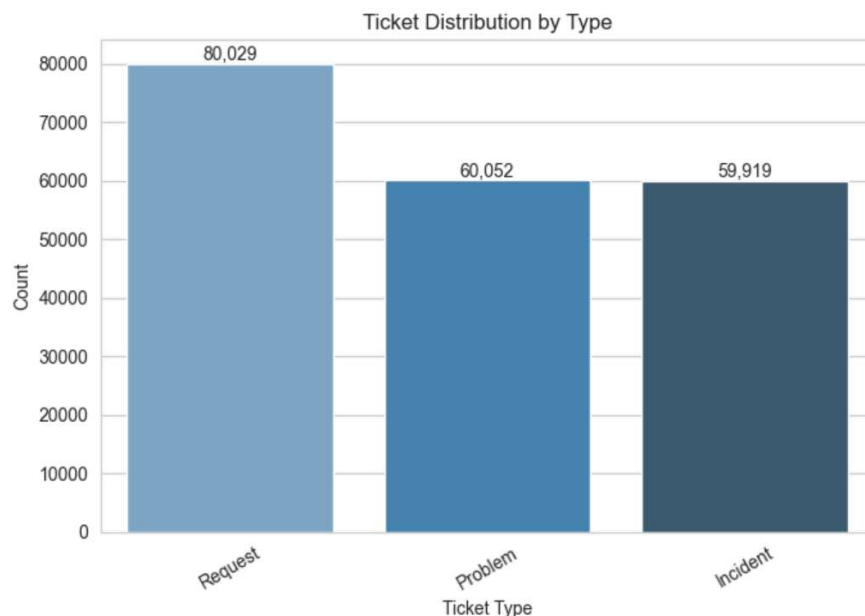
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style("whitegrid")
df = pd.read_csv("./cleaned_customer_support_tickets_200k.csv")
```

The required Python libraries, including **Pandas**, **Matplotlib**, and **Seaborn**, were imported for data analysis and visualization. Seaborn was configured with a whitegrid style to improve the appearance and readability of the charts. The cleaned customer support dataset was then loaded into a Pandas DataFrame using `pd.read_csv()` so that it could be used for further **exploratory data analysis and visualization**.

1. Ticket Type Distribution

Code:

```
plt.figure(figsize=(7,5))
ax = sns.countplot(data=df, x="type", order=df["type"].value_counts().index, palette="Blues_d")
plt.title("Ticket Distribution by Type")
plt.xlabel("Ticket Type")
plt.ylabel("Count")
plt.xticks(rotation=30)
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f"{int(x):,}")
plt.tight_layout()
plt.savefig("./images/M3/plot_ticket_type_distribution.png", dpi=200)
plt.show()
```



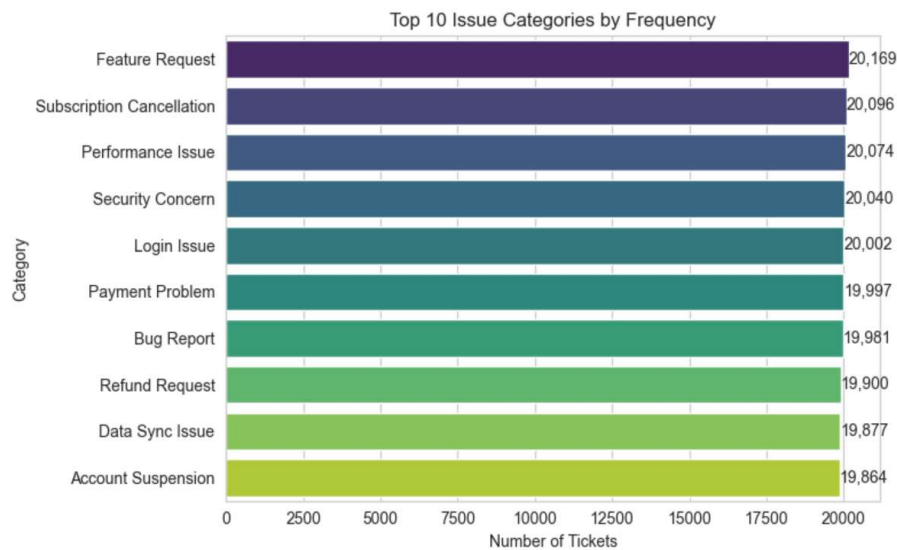
A **count plot** was created to visualize the number of tickets in each **ticket type**, such as Request, Problem, and Incident. The `value_counts()` function was used to arrange the ticket types from highest to lowest frequency, making it easier to compare their distribution.

The chart was customized with labels, a title, and ticket counts displayed on top of each bar. Finally, the visualization was saved as a PNG image for use in the project report.

2. Top Issue Categories by Frequency

Code:

```
plt.figure(figsize=(8,5))
top_categories = df["category"].value_counts().head(10)
ax = sns.barplot(x=top_categories.values, y=top_categories.index, palette="viridis")
plt.title("Top 10 Issue Categories by Frequency")
plt.xlabel("Number of Tickets")
plt.ylabel("Category")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f'{int(x):,}')
plt.tight_layout()
plt.savefig("./images/M3/plot_top_categories.png", dpi=200)
plt.show()
```



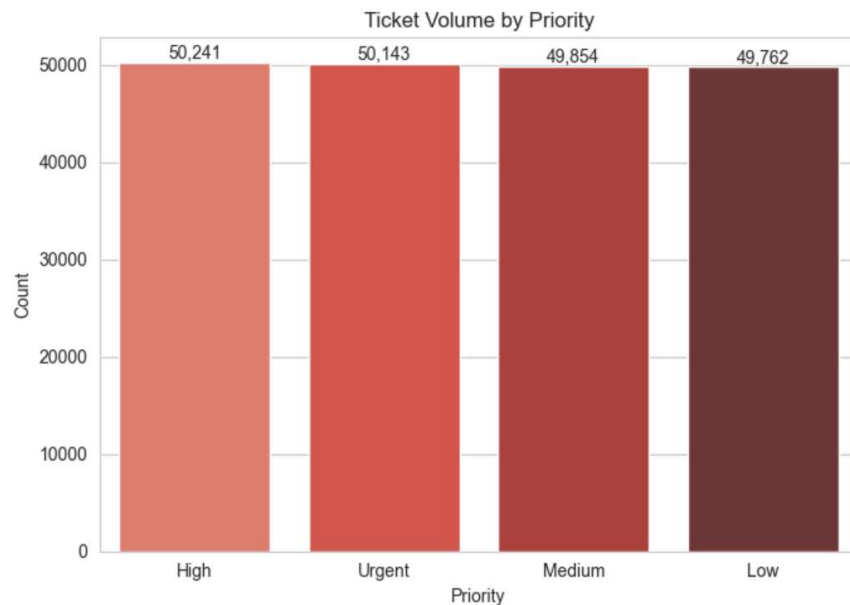
A horizontal **bar chart** was created to identify the **top 10 issue categories** based on the number of tickets. The `value_counts()` function was used to calculate the frequency of each category, and `.head(10)` selected the 10 most frequent categories.

The chart displays the categories in descending order, with the **number of tickets shown on each bar**. This makes it easier to identify the most common customer issues and understand where support teams receive the highest workload.

3. Tickets by Priority

Code:

```
plt.figure(figsize=(7,5))
ax = sns.countplot(data=df, x="priority", order=df["priority"].value_counts().index, palette="Reds_d")
plt.title("Ticket Volume by Priority")
plt.xlabel("Priority")
plt.ylabel("Count")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f'{int(x):,}')
plt.tight_layout()
plt.savefig("./images/M3/plot_priority_distribution.png", dpi=200)
plt.show()
```

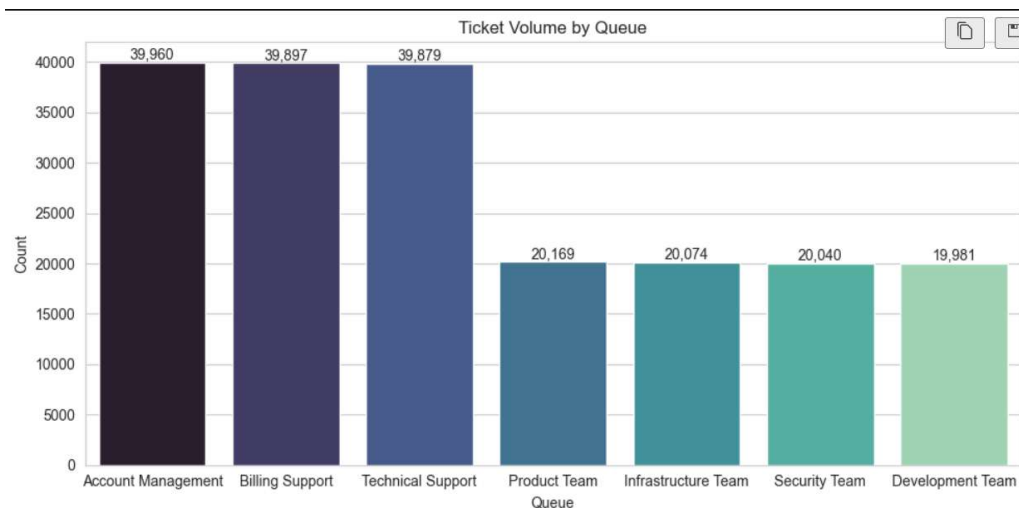


A count plot was created to show the distribution of customer support tickets across different priority levels, such as Low, Medium, High, and Urgent. The `value_counts()` function was used to arrange the priority levels based on their ticket frequency. The ticket count was displayed on top of each bar, making the comparison easier to understand. This visualization helps identify which priority levels contribute most to the overall support workload.

4. Tickets by Queue

Code:

```
plt.figure(figsize=(10,5))
ax = sns.countplot(data=df, x="queue", order=df["queue"].value_counts().index, palette="mako")
plt.title("Ticket Volume by Queue")
plt.xlabel("Queue")
plt.ylabel("Count")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f'{int(x):,}')
plt.tight_layout()
plt.savefig("./images/M3/plot_queue_distribution.png", dpi=200)
plt.show()
```



A count plot was created to visualize the number of tickets assigned to each support queue or team. The `value_counts()` function was used to arrange the queues according to their ticket volume, allowing easy comparison between different teams. The ticket count was displayed on each bar to clearly show the workload handled by each queue. This analysis helps identify which support teams are receiving the highest number of customer tickets.

Module 4

Similarity and Cluster Insights

- Analyze average similarity scores within clusters
- Compare cluster size vs. issue type
- Visualize performance gaps using boxplots and scatter plots
- Deliverables: Minimum 8 visualizations, Summary of key patterns and problem clusters

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import PCA

os.makedirs('images/M4', exist_ok=True)
sns.set_theme(style='whitegrid')
PALETTE = 'tab10'
N_CLUSTERS = 4
```

The required libraries were imported for **data processing, visualization, text analysis, and clustering**. TfidfVectorizer is used to convert ticket text into numerical features, while **K-Means** is used to group similar tickets into clusters.

The required output folder images/M4 was created to store visualizations, and Seaborn was configured for consistent chart styling. The value **N_CLUSTERS = 4** specifies that the customer support tickets will be divided into **four groups based on similarity**.

1. Load Data & Build Clusters

Code:

```
df = pd.read_csv('cleaned_customer_support_tickets_200k.csv')
df['issue_description'] = df['issue_description'].fillna("")
# TF-IDF vectorization
tfidf = TfidfVectorizer(max_features=500, stop_words='english')
X = tfidf.fit_transform(df['issue_description'])
# KMeans clustering into 4 clusters
km = KMeans(n_clusters=N_CLUSTERS, random_state=42, n_init=10)
df['cluster'] = km.fit_predict(X)
df['cluster_label'] = df['cluster'].map(lambda c: f'Cluster {c}')
print(df['cluster_label'].value_counts())
```

```
cluster_label
Cluster 0    99695
Cluster 1    40183
Cluster 3    39890
Cluster 2    20232
Name: count, dtype: int64
```

The cleaned customer support dataset was loaded, and missing values in the **issue_description** column were replaced with empty strings to ensure the text data could be processed without errors. **TF-IDF vectorization** was then used to convert the issue descriptions into numerical features based on the importance of words.

Next, the **K-Means clustering algorithm** grouped the tickets into **4 clusters** based on the similarity of their issue descriptions. A cluster label was assigned to each ticket, and the number of tickets in each cluster was displayed to understand the distribution of the groups.

2. Compute Cosine Similarity Scores

Code:

```
# Sample 5000 rows for similarity computation (memory-safe)
sample = df.sample(5000, random_state=42).copy()
X_sample = tfidf.transform(sample['issue_description'])

# Cosine similarity of each doc to its cluster centroid
centroids = km.cluster_centers_
sim_scores = []
for i, row in enumerate(X_sample):
    c = sample.iloc[i]['cluster']
    score = cosine_similarity(row, centroids[c].reshape(1, -1))[0][0]
    sim_scores.append(score)

sample['similarity_score'] = sim_scores

# Merge similarity scores back to full df (NaN for non-sampled rows)
df = df.merge(sample[['ticket_id', 'similarity_score']], on='ticket_id', how='left')

avg_sim = sample.groupby('cluster_label')['similarity_score'].mean().reset_index()
print(avg_sim)
```

	cluster_label	similarity_score
0	Cluster 0	0.447201
1	Cluster 1	0.742350
2	Cluster 2	1.000000
3	Cluster 3	0.753322

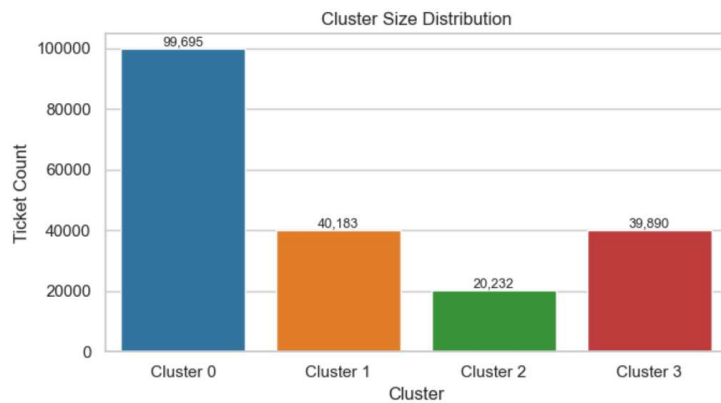
A sample of 5,000 tickets was selected to calculate similarity scores efficiently without using excessive memory. The Cosine Similarity method was used to measure how closely each ticket's issue description matches the centroid of its assigned cluster, with a higher score indicating greater similarity.

The calculated similarity scores were added to the sampled tickets and merged back into the main dataset. Finally, the average similarity score for each cluster was calculated to evaluate how well the tickets within each cluster are grouped together.

Plot 1 – Cluster Size Distribution (Bar)

Code:

```
fig, ax = plt.subplots(figsize=(7, 4))
counts = df['cluster_label'].value_counts().sort_index()
sns.barplot(x=counts.index, y=counts.values, palette=PALETTE, ax=ax)
ax.set_title('Cluster Size Distribution')
ax.set_xlabel('Cluster'); ax.set_ylabel('Ticket Count')
for p in ax.patches:
    ax.annotate(f'{int(p.get_height()):,}', (p.get_x() + p.get_width()/2, p.get_height()),
               ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.savefig('images/M4/plot1_cluster_size.png', dpi=150)
plt.show()
```

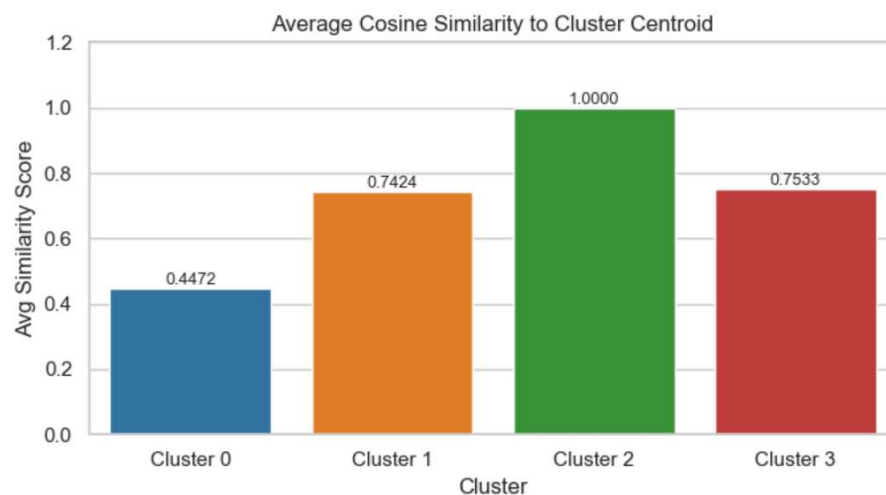


A **bar chart** was created to visualize the number of customer support tickets assigned to each of the **four clusters**. The `value_counts()` function was used to calculate the ticket count for each cluster, making it easy to compare the size of the groups. The exact ticket count was displayed above each bar for better readability. This visualization helps determine whether the tickets are **evenly distributed across clusters or concentrated in particular groups**.

Plot 2 – Average Similarity Score per Cluster (Bar)

Code:

```
fig, ax = plt.subplots(figsize=(7, 4))
sns.barplot(data=avg_sim, x='cluster_label', y='similarity_score', palette=PALETTE, ax=ax)
ax.set_title('Average Cosine Similarity to Cluster Centroid')
ax.set_xlabel('Cluster'); ax.set_ylabel('Avg Similarity Score')
ax.set_ylim(0, avg_sim['similarity_score'].max() * 1.2)
for p in ax.patches:
    ax.annotate(f'{p.get_height():.4f}', (p.get_x() + p.get_width()/2, p.get_height()),
                ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.savefig('images/M4/plot2_avg_similarity.png', dpi=150)
plt.show()
```



A **bar chart** was created to compare the **average cosine similarity score** of each cluster. The similarity score measures how closely the issue descriptions in a cluster match its cluster centroid, helping evaluate the consistency of the grouped tickets. The average score for each cluster was displayed on top of the bars. **Higher similarity scores indicate that the tickets within that cluster are more similar to one another**, while lower scores suggest greater variation in the issues.

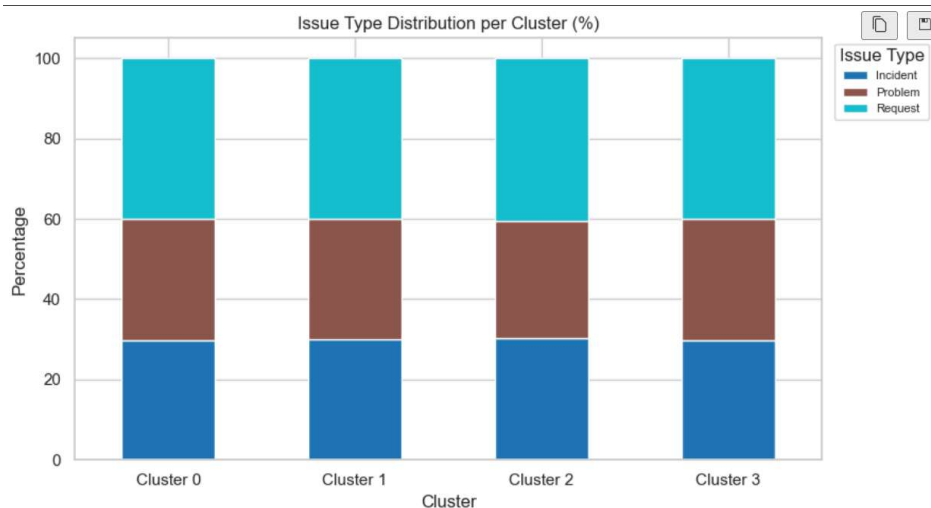
Plot 3 – Cluster vs Issue Type (Stacked Bar)

Code:

```
ct = df.groupby(['cluster_label', 'type']).size().unstack(fill_value=0)
ct_pct = ct.div(ct.sum(axis=1), axis=0) * 100
```



```
fig, ax = plt.subplots(figsize=(9, 5))
ct_pct.plot(kind='bar', stacked=True, colormap='tab10', ax=ax)
ax.set_title('Issue Type Distribution per Cluster (%)')
ax.set_xlabel('Cluster'); ax.set_ylabel('Percentage')
ax.legend(title='Issue Type', bbox_to_anchor=(1.01, 1), loc='upper left', fontsize=8)
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig('images/M4/plot3_cluster_issue_type.png', dpi=150)
plt.show()
```

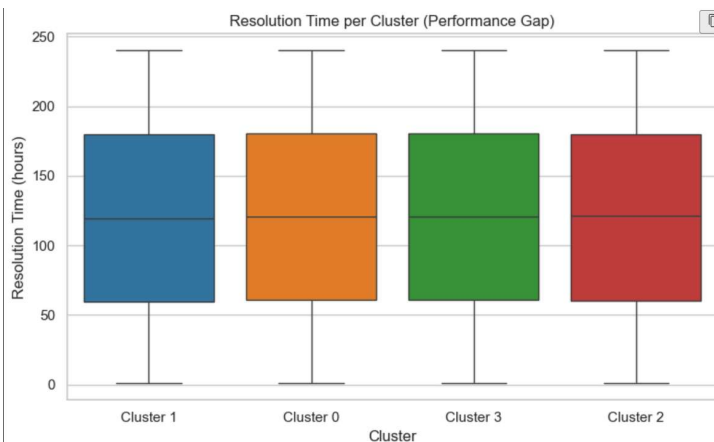


A **100% stacked bar chart** was created to compare the distribution of different **issue types** across the four clusters. The ticket counts were grouped by cluster and type, then converted into percentages so that each cluster represents 100%. This visualization helps identify the **dominant issue types within each cluster** and compare how the composition of Requests, Problems, and Incidents varies between clusters. It provides a clearer understanding of the types of customer issues represented by each cluster.

Plot 4 – Resolution Time Boxplot per Cluster (Performance Gap)

Code:

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=df, x='cluster_label', y='resolution_time_hours',
            palette=PALETTE, showfliers=False, ax=ax)
ax.set_title('Resolution Time per Cluster (Performance Gap)')
ax.set_xlabel('Cluster'); ax.set_ylabel('Resolution Time (hours)')
plt.tight_layout()
plt.savefig('images/M4/plot4_resolution_time_boxplot.png', dpi=150)
plt.show()
```



A **boxplot** was created to compare the distribution of **resolution time in hours** across the four clusters. The boxplot shows the median, spread, and overall variation in resolution time for each cluster, while extreme outliers were hidden using `showfliers=False`.

This visualization helps identify **performance gaps between clusters**. Clusters with higher median or wider distributions indicate tickets that generally take longer or have more variation in their resolution time.

Plot 5 – Customer Satisfaction Score Boxplot per Cluster

Code:

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=df, x='cluster_label', y='customer_satisfaction_score',
            palette=PALETTE, ax=ax)
ax.set_title('Customer Satisfaction Score per Cluster')
ax.set_xlabel('Cluster'); ax.set_ylabel('Satisfaction Score')
plt.tight_layout()
plt.savefig('images/M4/plot5_satisfaction_boxplot.png', dpi=150)
plt.show()
```



A **boxplot** was created to compare the distribution of **customer satisfaction scores** across the four clusters. The chart shows the median, spread, and variation of satisfaction scores within each cluster.

This visualization helps identify **which clusters have higher or lower customer satisfaction levels** and whether satisfaction varies significantly between different groups of support issues.

Milestone 3 - Week 5 & 6

Module 5

Performance Trend Analysis

- Analyze average resolution times by priority and type
- Determine which countries or teams handle tickets fastest
- Identify high-priority unresolved issues

Code:

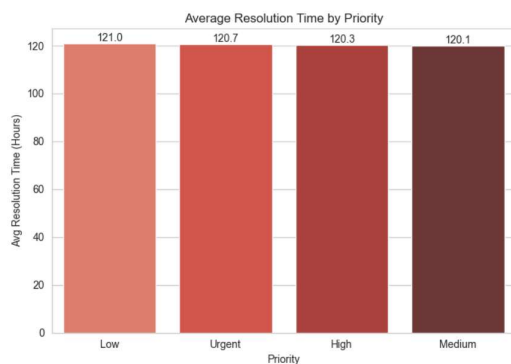
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
sns.set_style("whitegrid")
df = pd.read_csv("./cleaned_customer_support_tickets_200k.csv")
```

The required Python libraries were imported for data manipulation, numerical analysis, and visualization. Seaborn was configured with a whitegrid style to make the generated charts clear and readable. The cleaned customer support dataset was then loaded into a Pandas DataFrame using `pd.read_csv()`. This prepared the data for further analysis and visualization.

1. Average Resolution Time by Priority and Type

Code:

```
plt.figure(figsize=(7,5))
avg_by_priority = df.groupby("priority")["resolution_time_hours"].mean().sort_values(ascending=False)
ax = sns.barplot(x=avg_by_priority.index, y=avg_by_priority.values, palette="Reds_d")
plt.title("Average Resolution Time by Priority")
plt.xlabel("Priority")
plt.ylabel("Avg Resolution Time (Hours)")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f"{x:.1f}")
plt.tight_layout()
plt.savefig("./images/M5/plot_avg_resolution_by_priority.png", dpi=200)
plt.show()
```



The bar charts were created to analyze the average time required to resolve customer support tickets based on priority and ticket type.

By Priority: Tickets were grouped by priority level and their average resolution time was calculated. This helps identify which priority levels require more time to resolve.

2. Fastest Regions ("Countries") and Teams ("Queues")

Code:

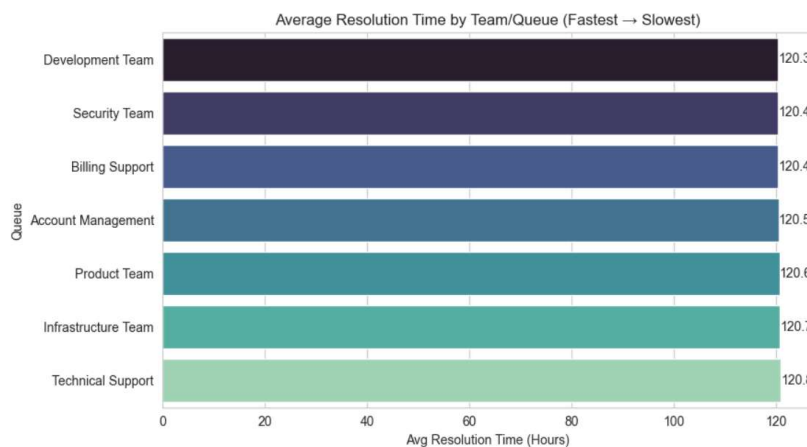
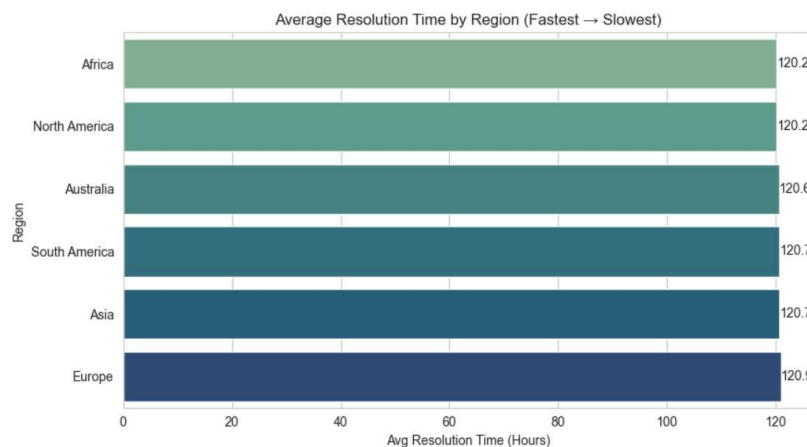
```
plt.figure(figsize=(9,5))
avg_by_region = df.groupby("region")["resolution_time_hours"].mean().sort_values()
```

```

ax = sns.barplot(x=avg_by_region.values, y=avg_by_region.index, palette="crest")
plt.title("Average Resolution Time by Region (Fastest → Slowest)")
plt.xlabel("Avg Resolution Time (Hours)")
plt.ylabel("Region")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f'{x:.1f}')
plt.tight_layout()
plt.savefig("./images/M5/plot_avg_resolution_by_region.png", dpi=200)
plt.show()

plt.figure(figsize=(9,5))
avg_by_queue = df.groupby("queue")["resolution_time_hours"].mean().sort_values()
ax = sns.barplot(x=avg_by_queue.values, y=avg_by_queue.index, palette="mako")
plt.title("Average Resolution Time by Team/Queue (Fastest → Slowest)")
plt.xlabel("Avg Resolution Time (Hours)")
plt.ylabel("Queue")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f'{x:.1f}')
plt.tight_layout()
plt.savefig("./images/M5/plot_avg_resolution_by_queue.png", dpi=200)
plt.show()

```



Two horizontal bar charts were created to compare the average resolution time across different regions and support queues.

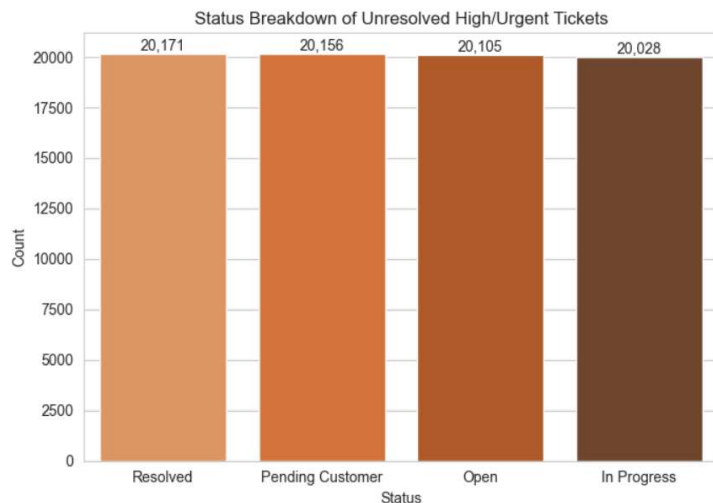
- By Region: The average resolution time was calculated for each region and sorted from fastest to slowest. This helps identify which regions resolve customer tickets more efficiently.
- By Queue: The average resolution time was calculated for each support queue and similarly arranged from fastest to slowest. This helps identify which teams handle and resolve tickets more efficiently.

3. High-Priority Unresolved Issues

Code:

```
unresolved_high_priority = df[
    (df["priority"].isin(["High", "Urgent"])) &
    (df["status"] != "Closed")
]
print(f"High/Urgent priority tickets still unresolved: {len(unresolved_high_priority):,}")

plt.figure(figsize=(7,5))
status_breakdown = unresolved_high_priority["status"].value_counts()
ax = sns.barplot(x=status_breakdown.index, y=status_breakdown.values, palette="Oranges_d")
plt.title("Status Breakdown of Unresolved High/Urgent Tickets")
plt.xlabel("Status")
plt.ylabel("Count")
for container in ax.containers:
    ax.bar_label(container, fmt=lambda x: f"{int(x):,}")
plt.tight_layout()
plt.savefig("./images/M5/plot_unresolved_high_priority.png", dpi=200)
plt.show()
```



The dataset was filtered to identify High and Urgent priority tickets that are still unresolved, excluding tickets with a Closed status. This helps highlight critical customer issues that may require immediate attention.

A bar chart was then created to show the status breakdown of these unresolved high-priority tickets. The visualization helps support teams quickly understand how many critical tickets remain in each unresolved status and prioritize them for faster resolution.

Module 6

Geographic and Category-Level Insights

- Create heatmaps of ticket concentration by region
- Map issue categories geographically using latitude/longitude
- Evaluate the relationship between cluster size and performance score
- Deliverables: Performance metrics summary, Geo-visualizations and comparative charts

Code:

```
sns.set_style("whitegrid")
os.makedirs("images/M6", exist_ok=True)
```

STEP 0: Confirm what 'region' actually contains

Code:

```
print("Unique regions:", df["region"].unique())
print("Number of regions:", df["region"].nunique())
```

```
Unique regions: <ArrowStringArray>
['North America', 'South America', 'Europe', 'Asia', 'Africa', 'Australia']
Length: 6, dtype: str
Number of regions: 6
```

The region column was examined to understand what values it contains and how many unique regions are present in the dataset. The unique() function displays all distinct region names, while nunique() calculates the total number of unique regions. This step ensures that the **region data is correctly structured and suitable for further regional analysis and visualization**.

1. Heatmap: Ticket Concentration by Region x Category

Code:

```
plt.figure(figsize=(12, 8))
region_category = pd.crosstab(df["region"], df["category"])
sns.heatmap(region_category, cmap="YlGnBu", annot=True, fmt="d", linewidths=0.5)
plt.title("Ticket Concentration by Region and Category")
plt.xlabel("Category")
plt.ylabel("Region")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig("images/M6/plot1_region_category_heatmap.png", dpi=150)
plt.show()
```



A **heatmap** was created to analyze the concentration of customer support tickets across different **regions and issue categories**. The pd.crosstab() function was used to calculate the number of tickets for each region-category combination. The heatmap uses color intensity to represent ticket volume, with **darker cells indicating a higher concentration of tickets**. This makes it easier to identify which issue categories are most common in each region and understand regional support patterns.

2. Geographic Mapping — region -> country centroid lookup

Code:

```
import plotly.express as px
```

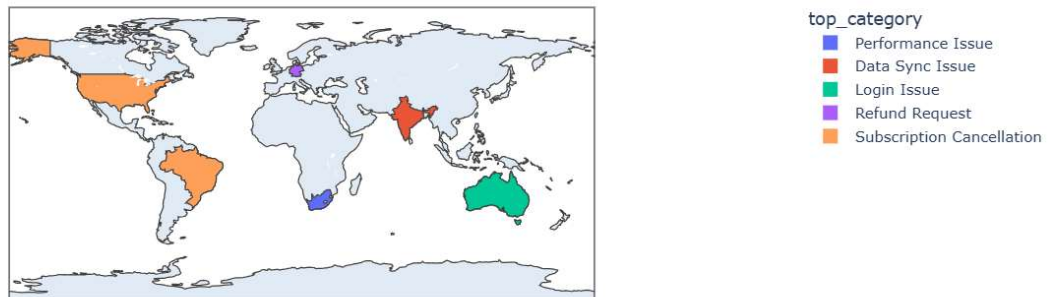
```
continent_map = {
    "Asia": "India",
    "Europe": "Germany",
    "North America": "United States",
    "South America": "Brazil",
    "Africa": "South Africa",
    "Australia": "Australia",
    "Oceania": "Australia"
}

region_summary = (
    df.groupby("region")
      .agg(
          ticket_count=("ticket_id", "count"),
          top_category=("category", lambda x: x.mode()[0])
      )
      .reset_index()
)

region_summary["country"] = region_summary["region"].map(continent_map)

fig = px.choropleth(
    region_summary,
    locations="country",
    locationmode="country names",
    color="top_category",
    hover_name="region",
    hover_data=["ticket_count"],
    title="Dominant Ticket Category by Region"
)
fig.write_html("images/M6/plot2_geo_category_map.html")
fig.show()
```

Dominant Ticket Category by Region



A **geographic choropleth map** was created to represent each region using a representative country. Since the dataset contains regional information rather than exact country locations, a country mapping was defined for each region, such as **Asia → India** and **Europe → Germany**.

The data was then grouped by region to calculate the **total ticket count** and the **most common ticket category**. The resulting map uses the dominant category to visualize regional differences and displays the ticket count when hovering over each region.

3. Cluster Size vs Performance Score

Code:

```
clustered = pd.read_csv("clustered_tickets.csv") # from Module 4 output
```

```

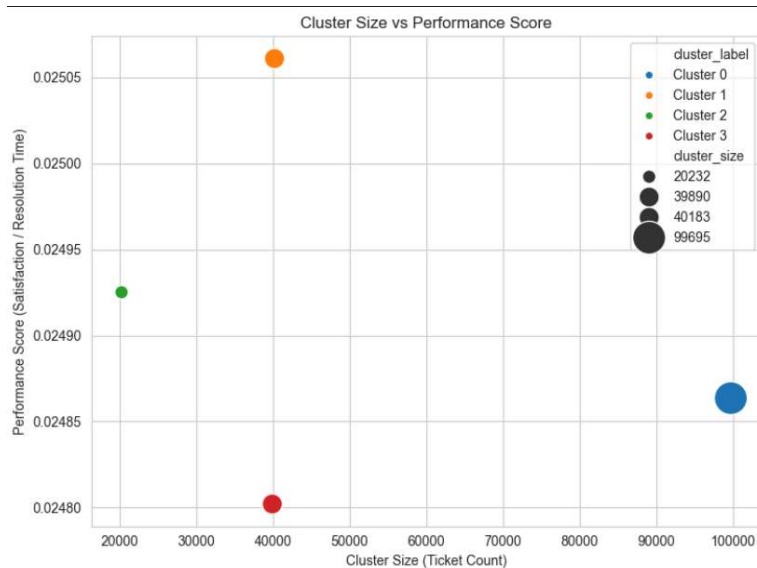
cluster_summary = clustered.groupby("cluster_label").agg(
    cluster_size=("ticket_id", "count"),
    avg_resolution_time=("resolution_time_hours", "mean"),
    avg_satisfaction=("customer_satisfaction_score", "mean"),
    avg_similarity=("similarity_score", "mean"),
).reset_index()

cluster_summary["performance_score"] = (
    cluster_summary["avg_satisfaction"] / cluster_summary["avg_resolution_time"]
)

plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=cluster_summary, x="cluster_size", y="performance_score",
    size="cluster_size", hue="cluster_label", palette="tab10",
    legend="full", sizes=(100, 600)
)
plt.title("Cluster Size vs Performance Score")
plt.xlabel("Cluster Size (Ticket Count)")
plt.ylabel("Performance Score (Satisfaction / Resolution Time)")
plt.tight_layout()
plt.savefig("images/M6/plot3_cluster_size_vs_performance.png", dpi=150)
plt.show()

print(cluster_summary)

```



```

cluster_label  cluster_size  avg_resolution_time  avg_satisfaction  \
0      Cluster 0      99695          120.753885          3.002377
1      Cluster 1      40183          119.705125          2.999925
2      Cluster 2      20232          120.475418          3.002867
3      Cluster 3      39890          120.877797          2.998020

avg_similarity  performance_score
0      0.447201          0.024864
1      0.742350          0.025061
2      1.000000          0.024925
3      0.753322          0.024802

```

The clustered dataset from **Module 4** was used to calculate the size and performance of each cluster. For every cluster, the **average resolution time**, **average customer satisfaction**, and **average similarity score** were calculated.

A **performance score** was then calculated as:

Performance Score = Average Satisfaction ÷ Average Resolution Time

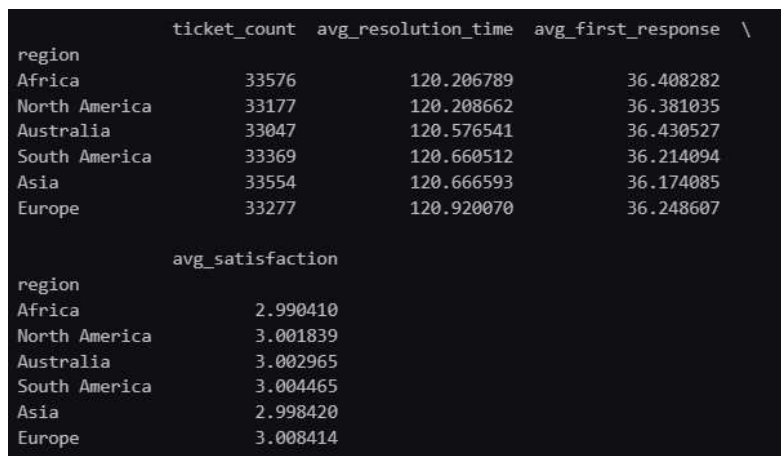
A **scatter plot** was created to examine the relationship between **cluster size and performance score**. This helps identify whether larger ticket clusters are associated with better or lower support performance. The cluster summary was also printed for detailed comparison.

4. Performance Metrics Summary (deliverable)

Code:

```
performance_summary = df.groupby("region").agg(
    ticket_count=("ticket_id", "count"),
    avg_resolution_time=("resolution_time_hours", "mean"),
    avg_first_response=("first_response_time_hours", "mean"),
    avg_satisfaction=("customer_satisfaction_score", "mean"),
).sort_values("avg_resolution_time")

performance_summary.to_csv("images/M6/performance_metrics_summary.csv")
print(performance_summary)
```



region	ticket_count	avg_resolution_time	avg_first_response
Africa	33576	120.206789	36.408282
North America	33177	120.208662	36.381035
Australia	33047	120.576541	36.430527
South America	33369	120.660512	36.214094
Asia	33554	120.666593	36.174085
Europe	33277	120.920070	36.248607

region	avg_satisfaction
Africa	2.990410
North America	3.001839
Australia	3.002965
South America	3.004465
Asia	2.998420
Europe	3.008414

A **performance summary by region** was created to provide an overall view of regional support performance. The analysis calculates the **ticket count, average resolution time, average first response time, and average customer satisfaction score** for each region.

The regions were sorted by **average resolution time**, from fastest to slowest, making it easier to compare regional performance. The final summary was exported as a **CSV file** for use as a project deliverable and further reporting.

5. Comparative Charts — Region Performance Comparison

Code:

```
plt.figure(figsize=(10, 6))
region_perf = df.groupby("region").agg(
    avg_resolution_time=("resolution_time_hours", "mean"),
    avg_satisfaction=("customer_satisfaction_score", "mean"),
).reset_index().sort_values("avg_resolution_time")
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
ax0 = sns.barplot(data=region_perf, x="avg_resolution_time", y="region", palette="crest", ax=axes[0])
axes[0].set_title("Avg Resolution Time by Region")
axes[0].set_xlabel("Hours")
axes[0].set_ylabel("Region")
for container in ax0.containers:
    ax0.bar_label(container, fmt=lambda x: f"{x:.1f}", fontsize=9, padding=3)

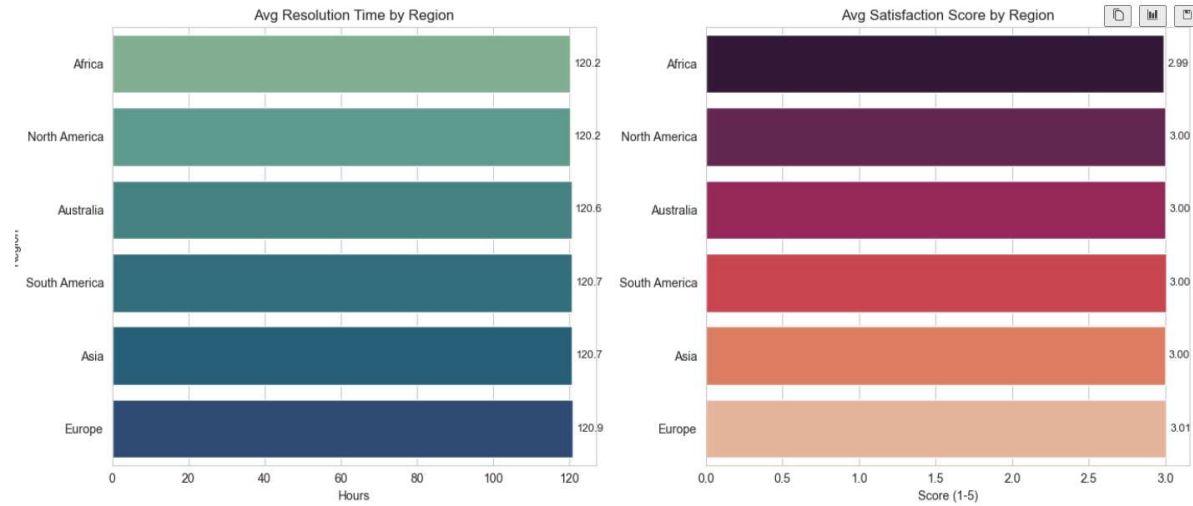
ax1 = sns.barplot(data=region_perf, x="avg_satisfaction", y="region", palette="rocket", ax=axes[1])
axes[1].set_title("Avg Satisfaction Score by Region")
axes[1].set_xlabel("Score (1-5)")
```

```

axes[1].set_ylabel("")
for container in ax1.containers:
    ax1.bar_label(container, fmt=lambda x: f"{x:.2f}", fontsize=9, padding=3)

plt.tight_layout()
plt.savefig("images/M6/plot4_region_comparison.png", dpi=150)
plt.show()

```



Two **horizontal bar charts** were created to compare regional performance using **average resolution time** and **average customer satisfaction score**. The regions were sorted by average resolution time to make performance differences easier to identify.

The first chart highlights which regions resolve tickets faster, while the second compares customer satisfaction across regions. Together, these charts provide a clear view of **regional efficiency and customer experience**.

Milestone 4 - Week 5

Module 7

Dashboard/Report Preparation

- Combine visual insights into an interactive Power BI or Streamlit dashboard
- Organize visualizations into performance KPIs: Average Resolution Time, Most Frequent Categories, Cluster Similarity Index, Top Performing Regions

Page 1: Executive Overview



Top numbers: 200K tickets total, ~120 hrs to resolve, satisfaction and SLA numbers look off/need checking.

Ticket Type: Requests are the biggest chunk (40%), Incidents and Problems split the rest evenly.

Trend Over Time: Ticket volume goes up and down each month, no clear pattern.

SLA Compliance: Exactly half of tickets breached SLA, half didn't.

Satisfaction by Channel: Every channel (chat, email, phone, etc.) scores the same — 3.0. No difference between them.

Ticket Status: Tickets are spread evenly across all statuses (Open, Closed, In Progress, etc.) — about 20% each.

Priority Level: High, Low, Medium, Urgent all have the exact same number of tickets (50K each).

Page 2: Category Performance



Ticket Volume by Category: All categories (Account Suspension, Bug Report, Data Sync, etc.) have exactly 20K tickets each — perfectly even.

Avg Resolution Time by Category: Every category takes about the same time, ~120–121 hours, no real difference between them.

Avg Resolution Time by Priority: High, Low, Medium, Urgent all take ~120–121 hours too — priority doesn't speed things up.

Queue Team Performance: Account Management, Billing Support, and Technical Support handle more tickets (~40K each)

than the other four teams (~20K each), but status mix (Open, Closed, etc.) is evenly split within each.

Product-wise Support Issues: Every product (Billing System, Cloud Storage, CRM, etc.) has the same 20K tickets — no product stands out as more problematic.

Page 3: Cluster Insights



Ticket Distribution Across Clusters: Cluster 0 is the biggest (~50% of all tickets), followed by Cluster 3 (20%), Cluster 2 (20%), and Cluster 1 (10%) — clusters are unevenly sized.

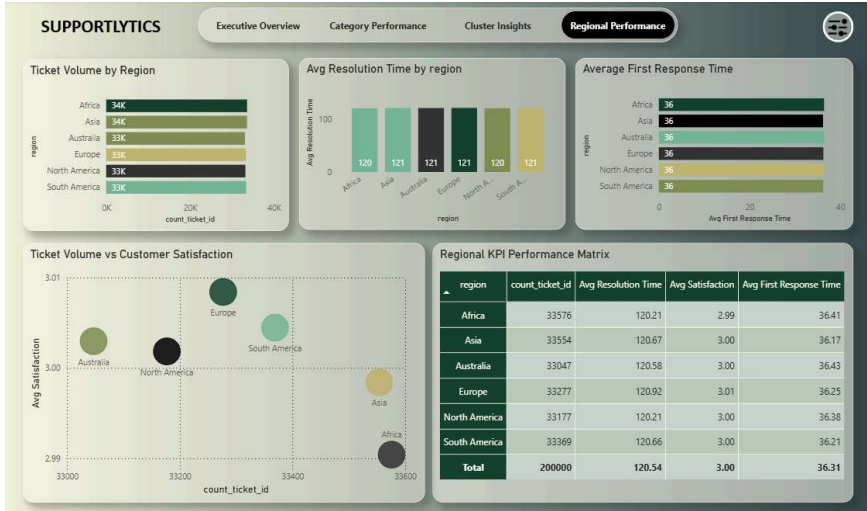
Avg Similarity Score by Cluster: Cluster 2 has the highest similarity (1.00, meaning tickets inside it are very alike), while Cluster 0 has the lowest (0.45, meaning it's the most mixed/diverse group).

Cluster Priority Distribution: Cluster 0 has the most tickets split evenly across all 4 priorities (25K each); Cluster 1 is smaller but still evenly split (10K each).

Cluster-wise Ticket Category Analysis: Cluster 0 has way more tickets across every category (~10K each) compared to Clusters 1, 2, and 3, which are much smaller.

Cluster Queue Performance: Cluster 0 dominates every team's workload (Account Management, Technical Support, Billing Support, etc.), while Clusters 1–3 contribute smaller, similar-sized shares.

Page 4: Regional Performance



Avg Resolution Time by Region: Resolution time is nearly identical everywhere (~120–121 hours), regardless of region.

Average First Response Time: All regions respond in about the same time too (~36 hours) — flat across the board.

Ticket Volume vs Customer Satisfaction: Satisfaction scores barely vary (2.99–3.01) across regions, and there's no real link between ticket volume and satisfaction — it's basically random scatter.

Regional KPI Matrix: Confirms everything above in one table — every region is nearly identical on volume, resolution time, satisfaction, and response time, with only tiny decimal-level differences.

Module 8

Documentation and Presentation

- Finalize project report (PDF/README)
- Prepare presentation slides highlighting insights and recommendations
- Upload code, visuals, and report to GitHub
- Deliverables: Final visual dashboard, Comprehensive project documentation, Presentation slides

1. Objectives

Module 8 focuses on consolidating the results from all previous modules and converting the analytical findings into **actionable recommendations for IT support management**. The final module brings together data analysis, clustering, performance evaluation, regional analysis, and visualization into a complete project outcome.

- Consolidate findings from all project modules.
- Identify the major factors affecting IT support performance.
- Compare performance across regions, priorities, issue types, queues, and clusters.
- Identify areas with high workload, slow resolution, and low customer satisfaction.
- Develop data-driven recommendations for improving support team efficiency.
- Finalize the dashboard, documentation, presentation, and project repository.

2. Final Performance Analysis

The final analysis combines the major performance metrics calculated throughout the project:

- Ticket Volume
- Average Resolution Time
- Average First Response Time
- Customer Satisfaction
- Priority Distribution
- Issue Category and Type
- Queue/Team Performance
- Regional Performance
- Cluster Performance
- High-Priority Unresolved Tickets

These metrics provide a comprehensive view of the current IT support operation and help identify areas requiring improvement.

3. Key Findings

Based on the analysis performed across the modules, the project identifies:

3.1 Ticket Workload

Ticket distribution analysis helps identify the **most frequently occurring issue types and categories**, allowing management to understand where support resources are being utilized most.

3.2 Priority Performance

The analysis of Low, Medium, High, and Urgent tickets helps determine whether **high-priority issues are being resolved efficiently** and whether critical tickets require additional attention.

3.3 Resolution Performance

Average resolution time was compared across **priority, ticket type, region, queue, and clusters** to identify areas where tickets take longer to resolve.

3.4 Support Team Performance

Queue-level analysis helps identify **faster and slower-performing support teams**, allowing resources and workloads to be distributed more effectively.

3.5 Regional Performance

Regional analysis compares **resolution time, first-response time, ticket volume, and customer satisfaction**, helping identify differences in support performance across regions.

3.6 Issue Clustering

K-Means clustering and TF-IDF text analysis grouped similar ticket descriptions into clusters. This helps identify **common patterns in customer issues** and supports better routing and categorization.

3.7 Customer Satisfaction

Customer satisfaction was compared across clusters and regions to identify areas where **longer resolution times or support processes may be associated with lower customer satisfaction.**

4. Recommendations

Based on the analytical results, the following recommendations can be proposed:

1. **Prioritize High and Urgent Tickets**
Create stronger monitoring and escalation mechanisms for unresolved high-priority tickets.
2. **Optimize Support Queues**
Redistribute workload based on queue performance and ticket volume to reduce bottlenecks.
3. **Improve First Response Time**
Introduce response-time targets and automated ticket assignment for faster initial responses.
4. **Use Automated Ticket Routing**
Use ticket categories and clustering results to automatically route similar issues to the appropriate support team.
5. **Monitor Regional Performance**
Continuously track regional resolution time, response time, and satisfaction to identify performance gaps.
6. **Focus on High-Volume Categories**
Analyze frequently occurring issue categories and develop FAQs, knowledge-base articles, or automated solutions for repetitive problems.
7. **Improve Customer Satisfaction**
Investigate clusters and queues with lower satisfaction scores and identify the operational causes behind them.
8. **Implement Continuous Performance Monitoring**

5. Final Dashboard

The final **Supportlytics dashboard** brings together the major KPIs and analytical insights into an interactive reporting interface.

The dashboard can provide:

- Total Ticket Volume
- Average Resolution Time
- Average First Response Time
- Average Customer Satisfaction
- Tickets by Priority
- Tickets by Category
- Tickets by Type
- Queue Performance
- Regional Performance
- Cluster Analysis
- High-Priority Unresolved Tickets

This dashboard enables IT support managers to quickly identify **performance gaps, workload concentrations, and areas requiring immediate action.**

6. Final Deliverables

The completed project will contain the following deliverables:

Deliverable	Description
Cleaned Dataset	Processed and feature-engineered customer support dataset
Python Code	Complete data cleaning, analysis, clustering, and visualization code
Analytical Visuals	Charts covering ticket, cluster, performance, and regional analysis

Power BI Dashboard	Interactive dashboard for monitoring support KPIs
Performance Summary	Regional and team-level performance metrics
Project Report	Comprehensive documentation of methodology and findings
Presentation	Slides covering objectives, methodology, insights, and recommendations
GitHub Repository	Central repository containing code, visuals, and documentation

Final Outcome

DV – Optimizing IT Support Team Performance Using Analytics (Supportlytics)

The **Supportlytics** project successfully transforms raw IT support ticket data into meaningful **performance insights and actionable recommendations**. Through data cleaning, feature engineering, exploratory analysis, text clustering, performance analysis, and visualization, the project provides a complete analytical view of IT support operations.

The final solution helps organizations:

- Identify high-volume support issues
- Detect performance bottlenecks
- Monitor resolution and response times
- Compare regional and team performance
- Identify unresolved high-priority tickets
- Understand customer satisfaction patterns
- Group similar support issues automatically
- Optimize ticket routing and resource allocation
- Improve overall IT support efficiency and customer experience

Final Project Impact

Supportlytics enables IT support leadership to move from reactive ticket handling to proactive, data-driven performance management.